MRM-Bench: Two-Page Research Pilot Plan

Assumption-Grounded Executable Verification of Credit Risk Models Using LLM Agents

Dushyant Pratap Singh | M.S. Data Science, University of North Texas | Research advisor: Dr. Chih-Hao Ku | October 2026

This pilot examines the reliability of AI-assisted verification of financial risk models; it is not an operational mortgage-approval system. MRM-Bench pairs deliberately faulty credit-risk model code with similar-looking valid code, each with an independently defensible answer key, to test whether an AI validation agent can detect genuine mathematical and implementation errors while reporting fewer false defects.

> **Research question:** *"At matched defect-detection recall, can assumption-applicability checks and independent verification reduce false-positive findings?"*

1. Dataset and feasibility (completed)

I downloaded the Freddie Mac Single-Family Loan-Level Dataset 2015 sample vintage (Release 47, July 2026) with its user guide, file layout (data dictionary) and headers. It has 50,000 fixed-rate loans (31 origination fields) and 3,467,166 monthly records (35 fields) for January 2015–March 2026; loan IDs match across files and no loan-month repeats. Key fields are credit score, original DTI and LTV, loan amount, interest rate, reporting period, delinquency status and zero-balance (termination) code. Documented "Not Available" codes become missing (FICO 9999; LTV, CLTV, DTI, MI 999; units, borrowers 99); DTI is 999 for 4,020 loans, 4,017 of them carrying the HARP (Relief Refinance) flag. Calendar coverage supports every planned horizon (each December 2015–2024 window, 12 months or shorter, ends before the March 2026 cutoff), but individual loans can still have incomplete follow-up (early exit or a gap; Section 2). Loan-level records stay on my machine; benchmark tests use synthetic data only.

2. Prediction date and 12-month outcome (implemented; awaiting your approval)

The prediction date t is the end of each December, 2015–2024. Features use only records dated ≤ t; the window is t+1 … t+12. Eligible loans have a record at t, no zero-balance code and status 00–02 (under 90 days past due).

- **Event (y = 1):** any window month with numeric status ≥ 03 or status RA, even if the loan later terminates.
- **Non-event (y = 0):** no event, and the loan has a record at t+12 or later (see the follow-up check).
- **Censored (counted, excluded from fitting and evaluation):** no event, and the loan's records end before t+12 through prepayment (01), another termination (02 third-party sale, 03 short sale or charge-off, 09 REO disposition, 15/16 loan sales, 96 defect) or the end of its data. Prepaid loans are never treated as negatives.

Numeric status ≥ 03 measures 90+ days past due; RA marks REO acquisition and is not a day count, so the implemented target is a composite. They nearly coincide: 2,489 of the 2,490 snapshot events have a numeric 90+ month, and the one RA-only event is a December 2016 training row. All 118 credit-related exits (codes 02, 03, 09) had a 90+ or RA month first, so none is censored. Prepayment censors 6.5–26.9% of at-risk loans per snapshot, hence a sensitivity check. This is a 12-month 90+ DPD research label, not a regulatory default definition.

Follow-up check (validation item). The non-event rule tests the last record, not every window month. Exactly one of 237,720 non-event rows (December 2015, training) has a gap: its t+12 record is missing, though the loan is current (00) before and after. No status is unknown (no XX; RA is the only non-numeric code), and zero-balance codes appear only on final records. Requiring all 12 months would censor that row (training 101,842 rows, 462 events; test and stress unchanged); the code and reported results are unchanged pending decision 2.

3. Clean logistic regression baseline (completed; converged official results)

I fitted one L2-regularized logistic regression (scikit-learn, C = 1, no class weights), $p = 1/(1 + e^{-z})$ with $z = b_0 + \mathbf{x}'\mathbf{b}$, on 23 inputs: origination fields (e.g., FICO, LTV, DTI plus a missing-DTI flag, rate, log loan amount) and summaries known at t (status, loan age, worst status and 30+ months in t−11 … t, prior 90+, modification, disaster and assistance flags). Median imputation, scaling and encoding are fitted on training rows only. *Chronological split:* train on December 2015–2018 and test on December 2021–2024; the COVID-19 forbearance windows of December 2019–2020 form a stress set never used for fitting. *Loan-disjoint split:* SHA-256(loan ID) mod 100 < 70 fixes each loan's pool across all years, and test and stress use only the other loans, so no loan is in both training and test (asserted in code). A future-invariance test with a mutation check (4/4 pass) shows no feature reads data after t.

Set	Decembers	Rows	Events	Event rate	Mean pred.	ROC-AUC [95% CI]	PR-AUC	Brier (base rate)
Train	2015–2018	101,843	462	0.45%	0.45%	0.891 [0.873, 0.909]	0.256	0.00381 (0.00452)
Test	2021–2024	15,716	119	0.76%	0.45%	0.893 [0.848, 0.928]	0.301	0.00622 (0.00751)
Stress	2019–2020	12,445	422	3.39%	0.60%	0.733 [0.707, 0.758]	0.153	0.03207 (0.03276)

CIs: loan-clustered bootstrap, 1,000 resamples; all stored values reproduce within 10^{-9}. With 119 test events, CIs are wide. The model ranks test loans well but under-predicts their level (0.45% vs 0.76%): a calibration finding, not a code defect; the higher test event rate is not yet analyzed. *Numerical correction:* converged results (tolerance 10^{-8}; Newton's method agrees) replace the default-10^{-4} fit (test ROC-AUC 0.894, PR-AUC 0.307): discrimination is materially unchanged, but rare-category predictions moved up to 0.12.

4. MRM-Bench: four verified defect pairs (completed; a feasibility pilot)

Loan data contain no answer key for code defects, so I built one. Each case has a written specification (rules, math, provenance), an implementation, a label and synthetic-data tests with hand-derived expected values.

Pair	Faulty case	Valid twin (false-alarm trap)	Key synthetic counterexample	Tests failed
P1 Preprocessing	DTI code 999 used as a number	999 → missing, plus a flag and training-median imputation	DTI {10, 20, 30, 40, 999, 999}: median must be 25; faulty gives 35	faulty 7/27 valid 0/27
P2 Formula	$p = 1/(1 + e^{z})$, a sign error	$p = ½(1 + \tanh(z/2))$: unfamiliar but identical	σ(ln 3) must be 0.75; faulty gives 0.25, yet passes range and symmetry tests	faulty 5/25 valid 0/25
P3 Horizon	6-month window, 12-month spec	Byte-identical code under a documented 6-month spec	2 of 4 events found; exits in months 7 to 12 become non-events, not censored	faulty 5/15 valid 0/15
P4 Leakage	Lookback window not capped at month t	Window t−11 … t (month t is allowed)	Deleting or altering records after t changes the features	faulty 5/18 valid 0/18

All four runners matched every label on 25 October 2026 (only assertion failures count). In P3, validity depends on the applicable specification; with H = 12 alone the code passes all 15 tests, so it has no second (censoring) error.

5. Proposed validation comparisons (proposed; none run yet)

Method	Capability (M2–M5 each add one capability to the previous method)
M1 Deterministic tests	Fixed numerical and property-based checks (range, monotonicity, future invariance); no LLM
M2 Read-only LLM	Reads only code and specification under neutral names (no labels or answer-key tests); executes nothing
M3 LLM + retrieval	M2 plus retrieved equations, specification clauses and Freddie Mac documentation
M4 Executable reviewer	M3 plus writing and running its own Python tests in a sandbox
M5 VALID-Agent	M4 plus an applicability check (why the rule's assumptions hold here) and an expected result justified independently of the code; its fresh-process re-run shows reproducibility, not correctness. Not built yet.

6. Evaluation metrics (proposed)

Recall and false-positive rate (FPR) are the shares of faulty and valid cases flagged. **Primary metric: FPR at matched defect-detection recall.** Methods score each case (LLMs: share of five runs flagging it; repeats are not new cases); recall is matched on development cases and FPR reported on held-out cases (cf. PrimeVul's VD-Score [1]). Secondary: precision, pair-wise outcomes, counterexample validity, explanation accuracy, stability, cost. **The eight-case pilot is intended to demonstrate feasibility, not to establish statistically significant superiority.**

7. Related work and research gap

LLM code review raises false-positive and trust concerns in industry [2]; LLMs can filter most static-analysis false alarms [3]; code agents can write tests that fail before and pass after a fix [4]; on PrimeVul's vulnerable/patched pairs, LLM detectors were near random in the strictest settings [1]. SR 26-2 (replacing SR 11-7) expects validation to find model errors, with independent effective challenge; it applies to traditional statistical and quantitative models such as this baseline, while generative and agentic AI models, like the proposed validator, are outside its direct scope [5]. Okpala et al. applied LLM agent crews to such tasks [6]; Kapoor and Narayanan catalogued eight leakage types [7]. **Gap:** this pilot investigates whether explicit mathematical assumption-applicability checks and independently verified executable tests improve defect discrimination in credit-risk model implementations. Because labels follow written specifications and each defect has a valid twin, spotting obvious bugs is not enough.

8. Decisions I need from you, and next steps

1. **Outcome:** make strict numeric 90+ DPD primary, with the RA composite as a sensitivity run?
2. **Censoring:** keep early exits censored, with a non-event sensitivity run? Require all 12 window months for non-events (censors one row)? Keep disaster-flagged events (50 of 462 training, 27 of 119 test)?
3. **Split:** loan-disjoint chronological test as primary (119 events), all-loan out-of-time check (418)?
4. **Benchmark:** approve the four categories and their valid twins as the pilot answer key.
5. **Comparisons:** approve M1–M5 and FPR at matched recall, thresholds set on development cases.
6. **Expansion:** grow to about 40 independent cases before building the complete VALID-Agent?

Next milestone: finalize these decisions and the answer key; run M1–M4 this winter, M5 in February–March 2027.

References

[1] Y. Ding et al., "Vulnerability detection with code language models: How far are we?" in *Proc. IEEE/ACM ICSE*, 2025.
[2] F. S. Aðalsteinsson et al., "Rethinking code review workflows with LLM assistance: An empirical study," arXiv:2505.16339, 2025.
[3] X. Du et al., "Reducing false positives in static bug detection with LLMs: An empirical study in industry," arXiv:2601.18844, 2026.
[4] N. Mündler et al., "SWT-Bench: Testing and validating real-world bug-fixes with code agents," in *Proc. NeurIPS*, vol. 37, 2024.
[5] Federal Reserve Board, FDIC, and OCC, *Revised Guidance on Model Risk Management*, SR 26-2, Apr. 17, 2026 (supersedes SR 11-7, 2011).
[6] I. Okpala, A. Golgoon, and A. R. Kannan, "Agentic AI systems applied to tasks in financial services: Modeling and model risk management crews," arXiv:2502.05439, 2025.
[7] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, 2023.

AI use: Claude and Claude Code (Anthropic) assisted with literature review, drafting and code; all results come from code I ran.